NoSQL RUD, Aggregation & 64-Dim Vector Similarity Search

Course: DBMS Mini Project (UE24CS395) • **Institution:** PES University (2026–2027)
Team: Saatvik Gupta (PES1UG24CS395) & Satyam Goyal (PES1UG24CS424)

1. 64-Dimensional Semantic Vector Similarity Search

Implemented deterministic 64-dimensional dense semantic feature hashing in backend/app/core/vector.py operating 100% offline with zero external dependencies. Documents and queries are normalized to the unit hypersphere and compared via cosine similarity. Exposed at GET /api/search/semantic.

```
# Execution Output from scripts/test_vector_search.py:
[QUERY]: "stolen blue luxury BMW sedan hotel valet"
  -> Generated Query Vector: 64 floats (L2 Normalized)
  -> Top 3 Semantic Matches:
     #1 [Score: 0.3388 | 33%] "Formal Sworn Statement of Valet Supervisor Pooja Kulkarni" [Witness statements]
     #2 [Score: 0.2806 | 28%] "Slack Internal Channel Dump: Fintech Nexa IT Security Emergency" [Chat exports]
     #3 [Score: 0.2503 | 25%] "National Vahan Vehicle Registry API Query: KA-01-MJ-8821" [JSON API responses]

[QUERY]: "cryptocurrency illicit transfer laundered wallet"
  -> Generated Query Vector: 64 floats (L2 Normalized)
  -> Top Semantic Match:
     #1 [Score: 0.2268 | 22%] "Telecom Network Operational Audit: Vasanth Nagar Exchange" [Case: CASE-2026-003]
```

2. MongoDB RUD & Multi-Stage Aggregation Pipeline

```
// Aggregation Pipeline: Intelligence Distribution by Document Type
db.investigation_documents.aggregate([
  { $group: {
      _id: "$document_type",
      total_docs: { $sum: 1 },
      distinct_cases: { $addToSet: "$case_id" },
      avg_tags: { $avg: { $size: { $ifNull: ["$tags", []] } } } }
  },
  { $project: {
      document_type: "$_id", total_docs: 1,
      total_cases: { $size: "$distinct_cases" },
      avg_tags: { $round: ["$avg_tags", 1] }
  }},
  { $sort: { total_docs: -1 } }
]);
// Full-Text Search with Relevance Scoring:
db.investigation_documents.find(
  { $text: { $search: "crypto tumbler wallet" } },
  { score: { $meta: "textScore" }, title: 1, document_type: 1 }
).sort({ score: { $meta: "textScore" } }).limit(3);
```